CSE211s Introduction to Embedded Systems

Spring 2025

Sheet 1

Review of Binary System

1- Add the following binary and hexadecimal numbers:

10010 + 1100 11110	1011101 +1000000 10011101	10011 +1111101 10010000
0xA5 +0x83 0x128	0xC4 +0xDE 0x1A2	0x46 +0x62 0xA8

ANS: addition of hexadecimal numbers

https://www.youtube.com/watch?v=ZZqT_hPshsM

2- How many times the binary number 10000 is greater than 10?

ANS: $(1000)_2 = (8)_{10}$ times.

3- For the three systems [binary, decimal, hexa-decimal] convert each of the following numbers to the other two systems:

$00101110_2 = 0x2E = 46$

$720_{10} = 1011010000 = 0x2D0$

$FE0F_{16} = 1111111000001111 = 65039$

ANS: to better understand, you can watch those videos

https://www.youtube.com/watch?v=CnvFy75ks_A

https://www.youtube.com/watch?v=t_kA5KQxByc

https://youtu.be/QJW6qnfhC70?si=Yxehd-xiVDR7PrRm&t=389

4- What is the largest binary number that can be expressed with 12 bits? What is the equivalent decimal & Hexa-Decimal.

ANS: $(1111\ 1111\ 1111)_2$ - $2^{12} - 1 = 4095 = 0xFFF$

In a computer system that represents all integer quantities using two's complement form, the most significant bit has a negative place-weight. For an eight-bit system, the place weights are as follows:

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

5- **Given this place-weighting, convert the following eight-bit two's complement binary numbers into decimal form:**

01000101_2 = 69 01110000_2 = 112 11000001_2 = 193

10010111_2 = 151 01010101_2 = 85

Note: A leading 0 means positive - read the number normally. A leading 1 means negative - flip the remaining bits, read their value, and add ONE to the value. Make that value negative.

In an eight-bit digital system, where all numbers are represented in two's complement form, what is the largest (most positive) quantity that may be represented with those eight bits?

ANS: 0111 1111 = $2^8 - 1$ = $(127)_{10}$

What is the smallest (most negative) quantity that may be represented? Express your answers in both binary (two's complement) and decimal form.

ANS: 1000 0000 = 111 1111 + 1 = -2^7 = 128 -> $(-128)_{10}$

6- **Add the following eight-bit two's complement numbers together, and then convert all binary quantities into decimal form to verify the accuracy of the addition:**

10110111	00111101	11111011
+01110110	+00111011	+11111011

ANS: 0010 1101 (45)
Carryout without overflow.
Sum is correct.

10000001	01111011	01111111
+10010001	+00111101	+10000001

ANS: Examples for better understanding the concept. (Signed numbers!!)

- -39 + 92 = 53:

$\boxed{1}$	1		1	1				
	1	1	0	1	1	0	0	1
+	0	1	0	1	1	1	0	0
	0	0	1	1	0	1	0	1

Carryout without overflow. Sum is correct.

- 44 + 45 = 89:

		1		1	1			
	0	0	1	0	1	1	0	0
+	0	0	1	0	1	1	0	1
	0	1	0	1	1	0	0	1

No overflow nor carryout.

- 104 + 45 = 149:

	1	1		1				
	0	1	1	0	1	0	0	0
+	0	0	1	0	1	1	0	1
	1	0	0	1	0	1	0	1

Overflow, no carryout. Sum is not correct.

- -103 + -69 = -172:

$\boxed{1}$		1	1	1		1	1	
	1	0	0	1	1	0	0	1
+	1	0	1	1	1	0	1	1
	0	1	0	1	0	1	0	0

Overflow, with incidental carryout. Sum is not correct.

- -75 + 59 = -16:

		1	1	1	1	1	1	
	1	0	1	1	0	1	0	1
+	0	0	1	1	1	0	1	1
	1	1	1	1	0	0	0	0

No overflow nor carryout.

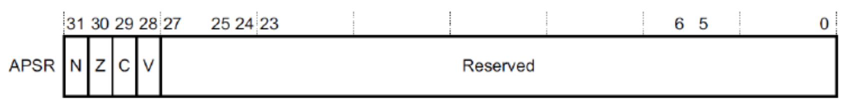
7- **How is it possible to tell that *overflow* has occurred in the addition of binary numbers, without converting the binary sums to decimal form and having a human being verify the answers?**
ANS: if the two numbers have the same sign (MSB) and the result comes with an opposite sign.

8- **Consider the addition of two signed 32-bit numbers. If a positive number is added to a negative number, under what conditions will a signed overflow occur?**
ANS: The overflow will not occur.

9- **Consider the subtraction of two signed 32-bit numbers. If a positive number is subtracted from another positive number, under what conditions will a signed overflow occur?**
An overflow will not occur. The smallest possible result is $R = 0 - 2^{N-1} = -2^{N-1}$
The Largest possible result $R = (2^{N-1} - 1) - 0 = 2^{N-1}$
R fits in N bits in both cases.

10- **Let A and B be two 8-bit inputs to an 8-bit binary adder. Fill in the table showing R=A+B and the four flags after each addition. The first row illustrates the process.**

ARMv6-M architecture Core Registers
Program Status Register (PSR)



- APSR condition code flags:
  1. N It is set if the result of the last mathematical operation is negative. It copies the leftmost bit in a two's complement representation.
  2. Z It is set if the result of the last mathematical operation is zero and reset otherwise.
  3. C Indicates that an arithmetic carry (or borrow) has been generated out of the most significant (ALU) bit.
  4. V: (Has meaning only with numbers that are defined as signed operands that can take positive and negative values) The overflow flag is set when the most significant bit (the sign bit) is changed by:
     - adding two numbers with the same sign or
     - subtracting two numbers with opposite signs.

     Overflow cannot occur when the sign of two addition operands are different (or the sign of two subtraction operands are the same)

ARM

Numbers are represented in two's complement in 8 bits. The result R must be kept 8 bits.

A	B	R_{10}	R_2	NZVC
10	100	110	0110 1110	0000
0x40 = $+64_{10}$ = $0100\ 0000_2$	0xA2 = -94_{10} = $1010\ 0010_2$	-30	1110 0010	1000
0xC3 = -61_{10} = $1100\ 0011_2$	0x6F = $+111_{10}$ = $0110\ 1111_2$	+50	0011 0010	0001
100	-100	0	0000 0000	0101
110 =01101110_2	146 = -110_{10} =10010010_2	0	0000 0000	0101
50	-50	0	0000 0000	**0101**

In the above examples, the V flag is never set because we are either adding two numbers of the same sign and the result has the same sign, or we are adding two number of opposite signs. In the last row you should deduce the value of B that would result in the given flags.

11- Let A and B be two 8-bit inputs to an 8-bit binary subtractor. Fill in the table showing R=A-B and the four flags after each subtraction. The first row illustrates the process. Calculate the carry bit in a similar way as the Cortex-M does.

A	B	-B	R_{10}	R_2	NZVC
100	10	$1111\ 0110_2$	90	0101 1010	0001
0x51 = $0101\ 0001_2$	0x93	$0110\ 1101_2$	190	1011 1110	1010
0xDF = $1101\ 1111_2$	0x9F	0110 0001	64	0100 0000	0001
-70 = 1011 1010	-70	0100 0110	0	0000 0000	0101
97	97	$1001\ 1111_2$	0	0000 0000	0101

You may insert a column in the table for -B in binary (2's complement) then perform the binary addition R=A+(-B)

12- Find the total amount of addressable memory, for each of the following CPUs, given the size of the address bus:

a) 16-bit address bus. (In Kilobytes)

ANS: 64 KB

1 KB = 2^{10} Bytes = 1024 Bytes, 1 MB = 1024 KB, 1 GB = 1024 MB, 1 TB = 1024 GB.

If an address bus is of size 16 bits, that means it can hold up to 2^{16} numbers and it hence can refer up to 2^{16} bytes of memory = 65,536 Bytes = 64 KB of memory and any memory greater than that is useless.

(Note: $2^{10} = 1\text{k}$, $2^{20} = 1\text{M}$, $2^{30} = 1\text{G}$, $2^{40} = 1\text{T}$).

$2^{16} = 2^6 \times 2^{10} = 64\text{KB}$

b) 24-bit address bus. (In megabytes)

ANS: 16 MB

Same idea

$2^{24} = 2^4 \times 2^{20} = 16\text{MB}$

c) 32-bit address bus. (In megabytes and gigabytes)

ANS: 4096 MB = 4 GB

Same idea

$2^{32} = 2^{12} \times 2^{20} = 4096\text{M} = 2^2 \times 2^{30} = 4\text{GB}$

d) 48-bit address bus. (In megabytes, gigabytes, and terabytes)

Same idea

$2^{48} = 2^8 \times 2^{40} = 256\text{TB}$

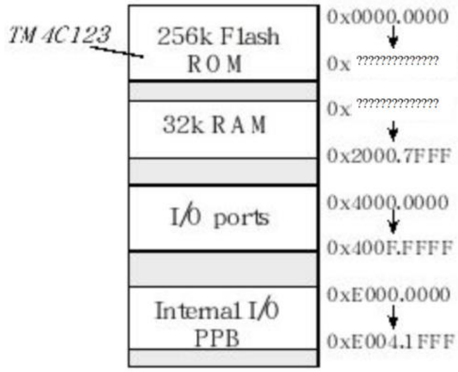
13- What components are usually put together with the microcontroller onto a single chip?

ANS: CPU, Input/output interfaces, Memory, Timers, …

14- In an embedded controller with on-chip ROM, why does the size of the ROM matter?

ANS: The ROM should have sufficient size to hold the code.

15- Given the memory map of the TM4C123:



a. **What is the ending address for 256K flash ROM?**

   ANS: 0x0003.FFFF

   1 KB = 1,024 Bytes, 256 KB = 262144 Bytes, thus 262,144-1 = $(262,143)_{10}$ = $(0x3.FFFF)_{16}$

b. **What is the starting address for 32K RAM?**

   ANS: 0x2000.0000

   32 KB = 32768 Bytes -> 32768-1 = $(32,767)_{10}$ = 0x7FFF -> 0x2000.7FFF – 7.FFF = 0x2000.0000

c. **What is the size of I/O ports registers block?**

   ANS:

   0x400F.FFFF – 0x4000.0000 = 0x000F.FFFF = 1,048,575 -> 1,048,575 + 1 = 1,048,576 Bytes

   1,048,576 / 1,024 = 1,024 KB -> 1,024 KB / 1,024 = 1 MB.

16- When signed overflow occurs?

When the result of the arithmetic operation does not fit in the available number of bits.
Assume N-bit precision:

- The range of non-negative numbers (sign bit = 0) is: $0 \xrightarrow{to} 2^{N-1} - 1.$
- The range of non-positive numbers (sign bit = 1) is: $-2^{N-1} \xrightarrow{to} 0.$
- Example: If N=8, then the range of signed numbers are $-2^7 \xrightarrow{to} 2^7 - 1.$

Different Cases for adding and subtracting two signed numbers A and B.

	$R = A + B$		$R = A - B$	
	Maximum $A_{max} + B_{max}$	Minimum $A_{min} + B_{min}$	Maximum $A_{max} - B_{min}$	Minimum $A_{min} - B_{max}$
A^+, B^+	$2^{N-1} - 1 + 2^{N-1} - 1 =$ $2^N - 2$ X	$0 + 0 = 0$	$2^{N-1} - 1 - 0 =$ 2^{N-1} X	$0 - (2^{N-1} - 1) =$ $-2^{N-1} + 1$
A^+, B^-	$2^{N-1} - 1 + 0 =$ $2^{N-1} - 1$	$0 + (-2^{N-1}) =$ -2^{N-1}	$2^{N-1} - 1 -$ $(-2^{N-1}) = 2^N - 1$ X	$0 - 0 = 0$
A^-, B^+	$0 + 2^{N-1} - 1 =$ $2^{N-1} - 1$	$-2^{N-1} + 0 =$ -2^{N-1}	$0 - 0 = 0$	$-2^{N-1} - (2^{N-1} - 1) =$ $-2^N + 1$ X
A^-, B^-	$0 + 0 = 0$	$-2^{N-1} + (-2^{N-1}) =$ -2^N X	$0 - (-2^{N-1}) =$ 2^{N-1} X	$-2^{N-1} - 0 =$ -2^{N-1}